ELTE | FACULTY OF INFORMATICS

AI Agents: From Concepts to Implementation

Open-Source Technologies for Real-Time Data Analytics

Pavlov Dusan
Teaching Assistant

Budapest, 2026

Lecture Overview

Part I - Understanding AI Agents

Concepts, Architectures & Fundamentals

- Fundamentals & architectures
- Function calling
- Reasoning and tool calling
- First AI agent

Part II - Building Agentic Systems

Implementation, Orchestration & Evaluation

- LangGraph fundamentals
- Stateful agent workflows
- Memory and control
- Evaluation and hands-on

Introduction

Why AI Agents?

Large Language Models (LLMs) are **powerful** at **understanding** and **generating** information. However, completing real-world tasks often requires interaction with external tools, data, and systems.

AI agents extend LLM capabilities by enabling them to:

- **Interact with the environment** - access external data, APIs, databases, and applications.
- **Make decisions and take actions** - dynamically select tools and determine the next steps.
- **Perform multi-step tasks** - coordinate several actions toward a defined goal.
- **Adapt to feedback** - use tool results to guide subsequent actions.

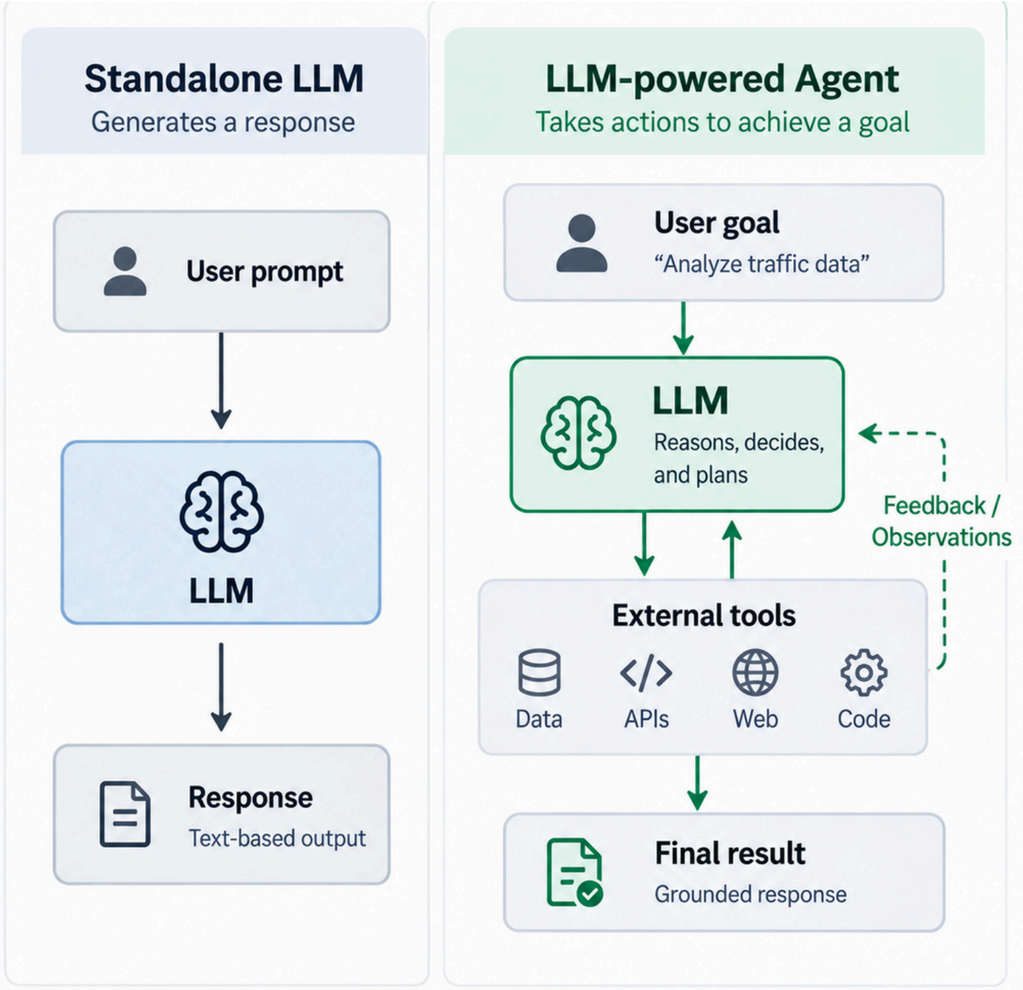
From User Requests to Actions

- **Goal**: The agent interprets the user's request and identifies the task to accomplish.
- **Decision**: Based on the available context, the agent determines the next appropriate action.
- **Execution**: The agent interacts with external tools, APIs, or data sources to perform operations.
- **Feedback**: Results are evaluated to decide whether further actions are needed.
- **Output**: The agent produces a final response based on the information collected.

LLMs vs AI Agents

What Makes an Agent Different?

- **LLM**: Generates responses based on prompts and available context.
- **AI Agent**: Uses an LLM to make decisions and interact with external tools.
- **Autonomy**: Selects actions dynamically rather than following a fixed sequence.
- **Goal-oriented**: Performs multiple steps and uses feedback to complete tasks.



AI Agent Reasoning

How does an **LLM determine** an agent's **next action**?

- **Input Context**: The model receives the user request, system instructions, conversation history, and available tool definitions.
- **Autoregressive Inference**: The LLM generates tokens conditioned on the current context, potentially producing a structured tool call instead of a natural-language response.
- **Tool Selection**: The model generates a tool name and arguments based on its learned behavior and the provided tool schemas.
- **Tool Execution**: The agent runtime validates and executes the requested function outside the LLM.
- **Observation & Iteration**: Tool results are added to the message history, and the model is invoked again to determine the next action.

Function Calling

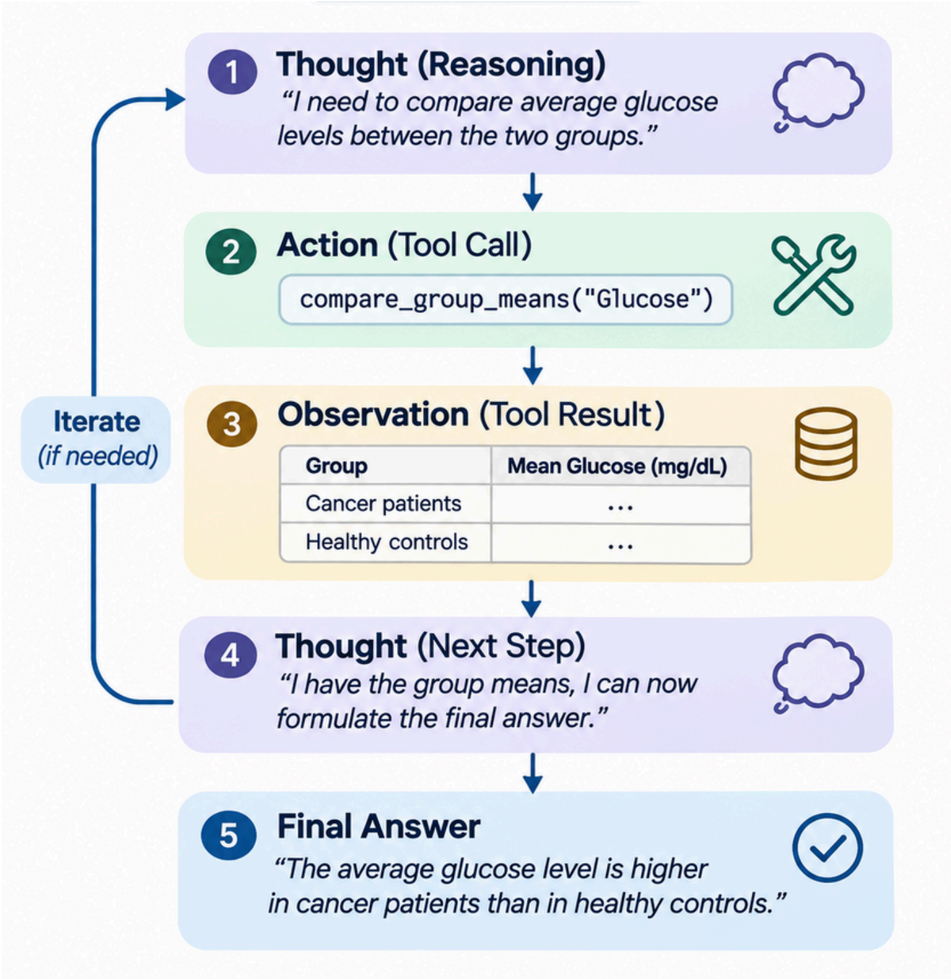
From model to execution:

- **LLM**: Interprets tasks, makes decisions, and selects actions.
- **Tools**: Enable access to external data, APIs, and executable functions.
- **Context** & **Memory**: Maintain task information, interaction history, and relevant observations.
- **Orchestration**: Manages execution, tool calls, state updates, and termination.

```
{
  "name": "compare_group_means",
  "arguments": {
    "feature": "Glucose"
  }
}
```

ReAct: Reasoning and Acting

- **ReAct**: Combines iterative decision-making with external actions.
- **Action**: The LLM requests a tool based on the current task.
- **Observation**: The tool result is returned to the model.
- **Iteration**: The model decides whether another action is required.
- **Termination**: Execution ends when a final response is produced or a limit is reached.



Ex 1. Building Our First AI Agent

Objective: Develop an LLM-powered agent that analyzes clinical data and answers natural-language questions using Python tools.

Task Requirements

1. **Dataset**: UCI Breast Cancer Coimbra - clinical measurements from cancer patients and healthy controls.
2. **LLM**: Use Ollama (Qwen3) for natural-language understanding and tool selection.
3. **Python Tools:** Implement three statistical functions:
   - get_cohort_summary() - Patient and control counts.
   - compare_group_means(feature) - Compare clinical measurements between groups.
   - get_feature_statistics(feature, group) - Compute descriptive statistics.
4. **Framework**: Integrate the LLM and tools using LangChain.
5. **Validation**: Inspect tool calls and verify results against the dataset.

Expected Outcome:

An AI agent capable of answering questions such as:

"*How many breast cancer patients are there, and how do their average glucose levels compare with healthy controls?*"

Deliverable: A reproducible Python application demonstrating LLM-driven tool selection and data-grounded analysis.

Environment Setup & Project Initialization

Prerequisites:

- Python 3.11/3.12
- **Ollama**: Run an LLM locally without paid APIs.
- **LangChain**: Agent orchestration and tool integration.
- **pandas**: Healthcare dataset analysis.
- **Dataset**: UCI Breast Cancer Coimbra.

```powershell
# 1. Create virtual environment
py -3 -m venv .venv

# 2. Install dependencies
.\.venv\Scripts\python.exe -m pip install
-r requirements.txt

# 3. Download dataset
.\.venv\Scripts\python.exe
download_data.py

# 4. Download LLM (Ollama installed)
ollama pull qwen3:1.7b
```

PowerShell

Defining Tools for the AI Agent

Agent Tools

- **Python Functions**: Statistical operations on the healthcare dataset.
- **Tool Decorator (@tool)**: Exposes functions, descriptions, and parameters to the LLM.
- **Available Tools**:
  - *get_cohort_summary*() - Patient counts.
  - *compare_group_means(feature)* - Compare clinical features between groups.
  - *get_feature_statistics(feature, group)* - Calculate descriptive statistics.
- **Execution**: The LLM selects tools, while Python performs the actual calculations.

```python
from langchain.tools import tool
from stats import load_data
from stats import get_cohort_summary
     as calculate

@tool
def get_cohort_summary() -> dict:
    """Count patients and healthy
    controls."""
    return calculate(load_data())
```

agent_tools.py

Initializing the LLM-Powered Agent

Agent Configuration

- **LLM**: Ollama hosts the Qwen3 model locally for inference and tool selection.
- **Tool Registration**: Python functions are provided to the agent as callable tools.
- **System Prompt**: Defines the agent's role, instructions, and limitations.
- **Agent Initialization**: LangChain's create_agent() connects the model, tools, and instructions.
- **Execution**: The framework manages iterative model calls and tool execution.

```python
from langchain.agents import create_agent
from langchain_ollama import ChatOllama
from agent_tools import TOOLS

model = ChatOllama(
    model="qwen3:1.7b",
    temperature=0
)

agent = create_agent(
    model=model,
    tools=TOOLS,
    system_prompt=SYSTEM_PROMPT
)
```

agent.py

Running and Inspecting the AI Agent

Agent Execution

- **User Query**: Submit a natural-language question to the agent.
- **Model Inference**: The LLM determines which statistical tools to invoke.
- **Tool Execution**: LangChain executes the selected functions and returns their results.
- **Execution Trace**: Inspect tool names, arguments, and observations.
- **Validation**: Verify that the final response is supported by the computed statistics.

```
question = "Compare mean glucose between
groups."

result = agent.invoke({
    "messages": [
        {"role": "user", "content": question}
    ]
})

show_tool_trace(result["messages"])
print(result["messages"][-1].content)
```

agent.py

```
[TOOL CALL]
compare_group_means({"feature": "Glucose"})

[TOOL RESULT]
{... group averages ...}

[ANSWER]
Comparison based on computed statistics.
```

console output

Hands-On: Testing and Extending the Agent

Tasks:

- **Test**: Ask the agent different questions about the clinical dataset.
- **Multi-Tool Execution**: Submit a question requiring multiple statistical calculations.
- **Extend**: Add a new tool for calculating median glucose levels by diagnosis group.
- **Inspect**: Examine the selected tools, arguments, and returned values.
- **Evaluate**: Verify numerical correctness and identify potential limitations.

TASK 1 — SINGLE TOOL
"*What is the average BMI of breast cancer patients*?"
Expected: Statistical tool selection

TASK 2 — MULTIPLE TOOLS
"*How many cancer patients are there, and how do their glucose levels compare with healthy controls*?"
Expected: Multiple tool calls

TASK 3 — EXTENSION
"*Compare median glucose levels between the two groups*."
Expected: Implement and register a new tool

Evaluating AI Agent Performance

Evaluation Criteria

- **Tool Selection Accuracy**: Does the LLM choose the appropriate function for each request?
- **Answer Correctness**: Are the generated conclusions consistent with computed statistics?
- **Robustness**: Can the agent handle ambiguous or unsupported queries?
- **Execution Efficiency**: How many tool calls and how much time are required?
- **Reliability**: Does the agent produce consistent results across repeated executions?

Example Evaluation

Test Case	Expected Behavior
Mean glucose by group	Correct tool and values
Multi-step query	Uses necessary tools
Unsupported feature	Explains limitation
Medical diagnosis request	Avoids diagnosis
Repeated question	Consistent numerical results

Additional LLM models for agent

Model alternatives:

- "**llama3.2:1b**" — Lightweight; moderate tool calling.
- "**granite4:350m-h**" — Ultra-light; limited reasoning.
- "**functiongemma**" — Tiny; tool-calling specialist.
- "**smollm2:360m**" — Tiny; unreliable tool calling.
- "**qwen3:0.6b**" — Lightweight; basic tool calling.
- "**qwen2.5:0.5b**" — Tiny; limited reliability.

OpenAI model

1. Open OpenAI API Keys.
2. Create a new secret key.
3. Check your available credits and billing settings at OpenAI API Billing.

```
pip install -U langchain-openai
export OPENAI_API_KEY="your-api-key"
```

terminal

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5-nano"
)
```

agent.py

Grok as online alternative for LLM models

Step 1 — Create a free GroqCloud account

1. Open GroqCloud Console.
2. Sign in using Google, GitHub, or email.
3. Open API Keys.
4. Click Create API Key and copy the key.

```
pip install -U langchain-groq
export GROQ_API_KEY="your-api-key-here"
```

terminal

```
from langchain_groq import ChatGroq

model = ChatGroq(
    model="openai/gpt-oss-20b",
    temperature=0
)
```

agent.py

Summary: Understanding AI Agents

Key Takeaways:

- **AI Agents**: LLM-powered systems capable of making decisions and interacting with external environments.
- **Agent Architecture**: Combines an LLM, tools, context, memory, and orchestration.
- **Reasoning & Acting (ReAct)**: Enables iterative decision-making through actions and observations.
- **Tool Calling**: LLMs generate structured tool requests, while the runtime executes the corresponding functions.
- **Practical Implementation**: Built a healthcare data-analysis agent using LangChain, Ollama, and Python.
- **Evaluation**: Assessed tool selection, numerical correctness, robustness, and reliability.

Looking Ahead — Lecture II

Building Reliable Agentic Systems with LangGraph

Part II
Building Agentic Systems

Introduction

Why **LangGraph**?

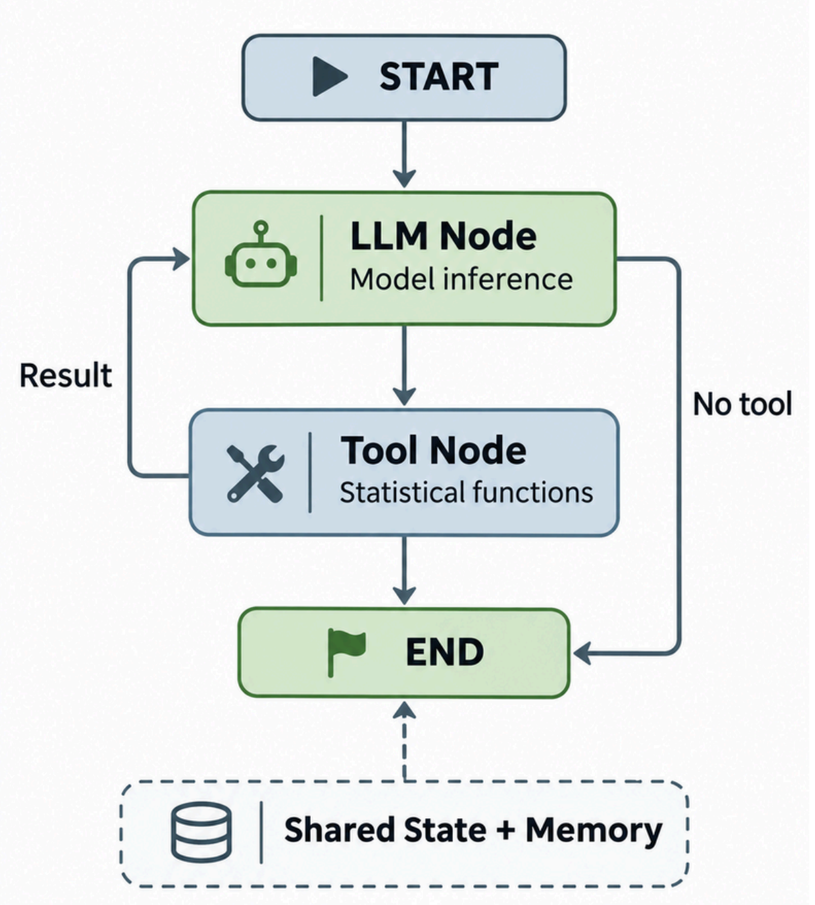
- **From Agents to Systems**: Extend our healthcare AI agent with explicit control over its execution.
- **Graph-Based Architecture**: Represent agent operations as nodes connected by edges.
- **State Management**: Maintain messages, tool results, and context throughout execution.
- **Conditional Execution**: Dynamically route between the LLM, external tools, and termination.
- **Persistence**: Preserve conversation state across multiple interactions.

Learning Objective: Understand how LangGraph enables controlled, stateful execution of LLM-powered agents.

LangGraph Architecture: State, Nodes and Edges

Core Components:

- **State**: Stores messages, context, and intermediate results.
- **Nodes**: Execute operations such as LLM inference or tool calls.
- **Edges**: Define execution flow between nodes.
- **Conditional Edges**: Route execution based on the current state.
- **START / END**: Mark graph entry and termination.
- **Checkpointer**: Persists state for memory and recovery.



Building the Agent Graph with LangGraph

Implementation:

- State: Use MessagesState to maintain conversation history.
- LLM Node: Invoke the model with the current messages and available tools.
- Tool Node: Use ToolNode to execute model-requested functions.
- Routing: Use tools_condition to determine whether to execute a tool or finish.
- Graph Compilation: Connect nodes and edges, then compile the executable graph.

```
from langgraph.graph import StateGraph,
MessagesState, START
from langgraph.prebuilt import ToolNode,
tools_condition

builder = StateGraph(MessagesState)

builder.add_node("llm", call_llm)
builder.add_node("tools", ToolNode(TOOLS))

builder.add_edge(START, "llm")
builder.add_conditional_edges("llm",
tools_condition)
builder.add_edge("tools", "llm")

graph = builder.compile()
```

langgraph_agent.py

Adding Conversation Memory with LangGraph

State Persistence:

- **Checkpointer**: Saves the agent's state between interactions.
- **InMemorySaver**: Stores conversation history in memory during execution.
- **Thread ID**: Identifies a conversation and links subsequent requests.
- **Context Retention**: Enables follow-up questions without repeating previous information.
- **Limitation**: In-memory state is lost when the application restarts.

```
from langgraph.checkpoint.memory import InMemorySaver

# Compile graph with memory
graph = builder.compile(
 checkpointer=InMemorySaver()
)
```

```
config = {
 "configurable": {"thread_id": "patient-
analysis-1"}
}

# First question
graph.invoke(
    {"messages": [
        {"role": "user",
         "content": "Compare mean glucose
between groups."}
    ]},
    config=config
)

# Follow-up using the same thread
result = graph.invoke(
    {"messages": [
        {"role": "user",
         "content": "What about insulin?"}
    ]},
    config=config
)
```

langgraph_agent.py

Hands-On: Testing Stateful Agent Behavior

Tasks

- Run: Execute the LangGraph agent with conversation memory enabled.
- Test Memory: Ask related questions using the same thread_id.
- Compare: Change the thread_id and observe how the agent responds without previous context.
- Inspect: Examine tool calls, arguments, and stored conversation history.
- Evaluate: Verify statistical results and discuss the benefits and limitations of memory.

Experiment 1 - SAME CONVERSATION:

- **Question 1**: "Compare mean glucose levels between cancer patients and healthy controls."
- **Question 2**: "What about insulin?"

Experiment 2 - NEW CONVERSATION:

- **Question**: "*What about insulin*?"
- ***Expected***: Without previous context, the request is ambiguous.

Experiment 3 - STATE INSPECTION:

```
state = graph.get_state(config)
print(state.values["messages"])
```

Summary: Agentic Systems with LangGraph

Key Takeaways

- **LangGraph**: Enables explicit control over agent execution through graph-based workflows.
- **State Management**: MessagesState maintains conversation messages and tool results.
- **Graph Architecture**: Nodes execute operations, while edges determine execution flow.
- **Tool Integration**: ToolNode and tools_condition enable iterative LLM–tool interactions.
- **Conversation Memory**: Checkpointing preserves context across requests using thread_id.
- **Practical Implementation**: Extended our healthcare AI agent with stateful execution and conversational memory.

Further Directions

- **Human-in-the-Loop**: Introduce approval steps for sensitive actions.
- **Persistent Memory**: Use database-backed checkpointing.
- **Evaluation**: Measure tool-selection accuracy, latency, and reliability.



ELTE | FACULTY OF INFORMATICS

Thank you for your attention!

Pavlov Dusan

Teaching Assistant

Open-Source Technologies for Real-Time Data Analytics, Budapest, 2026.